Data Wrangling for Visualization (Python)

Workshop 4 — Module 3

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

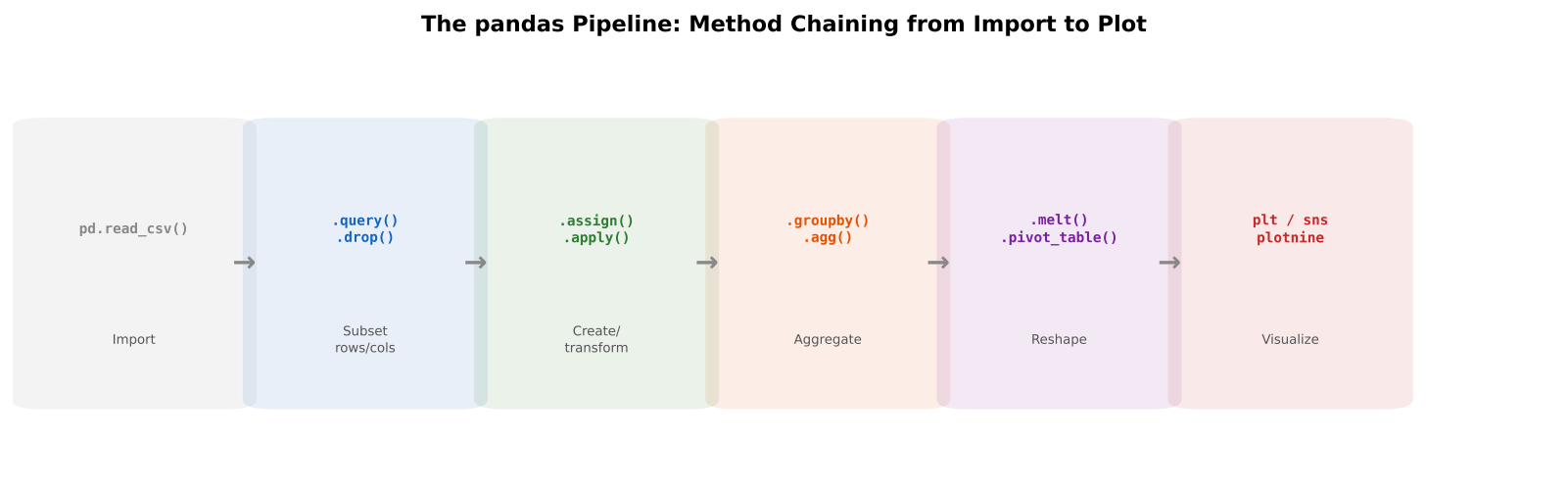
1. The pandas pipeline: method chaining as Python's pipe
2. R $\leftrightarrow$ Python Rosetta Stone: every dplyr verb mapped
3. .assign(), .query(), .groupby().agg(): the core trio
4. Reshaping with pd.melt() and .pivot_table()
5. Common cleaning patterns in pandas
6. Full chain: one expression from CSV to chart

Learning Outcome

You will translate any R/dplyr wrangling pipeline into idiomatic pandas method chains, and produce the same ggplot-ready data structures using Python's .groupby().agg() and pd.melt().

The pandas Pipeline

Method Chaining: Python's Pipe



Chaining vs Temporary Variables

Method Chaining: pandas Equivalent of R's Pipe

WITHOUT chaining

```
temp1 = df[df['Rating'] > 4]
temp2 = temp1.groupby('Cat')
temp3 = temp2['Rating'].mean()
temp4 = temp3.nlargest(8)
temp4.plot.barh()
```

WITH chaining

```
(df
.query('Rating > 4')
.groupby('Category')
['Rating'].mean()
.nlargest(8)
.plot.barh())
```

Pro-Tip

Wrap the entire chain in parentheses so Python allows multi-line method calls without backslash continuation. The `.assign()` method creates columns inline (like dplyr's `mutate()`). Use `lambda d:` inside `.assign()` to reference the DataFrame being transformed.

R $\leftrightarrow$ Python Rosetta Stone

Every dplyr Verb → pandas Equivalent

R ↔ Python Wrangling Rosetta Stone

Task	R (dplyr/tidyr)	Python (pandas)
Subset rows	`filter(Rating > 4)`	`df.query('Rating > 4')` `df[df['Rating'] > 4]`
Subset cols	`select(App, Rating)`	`df[['App', 'Rating']]` `df.filter(['App','Rating'])`
Create cols	`mutate(log_r =` `  log10(Reviews))`	`df.assign(log_r =` `  np.log10(df['Reviews']))`
Sort rows	`arrange(desc(Rating))`	`df.sort_values('Rating',` `  ascending=False)`
Group + Agg	`group_by(Cat) |>` `  summarise(m = mean(R))`	`df.groupby('Cat')['R']` `  .mean()  # or .agg()`
Count	`count(Category, sort=T)`	`df['Category']` `  .value_counts()`
Reshape long	`pivot_longer(cols,` `  names_to, values_to)`	`pd.melt(df,` `  id_vars, var_name, value_name)`
Reshape wide	`pivot_wider(names_from,` `  values_from)`	`df.pivot_table(index,` `  columns, values)`
Join	`left_join(x, y, by='k')`	`pd.merge(x, y, on='k',` `  how='left')`

.query() + Indexing: Subset Rows and Columns

```
# .query(): string-based filtering (like dplyr::filter)
df.query("Rating > 4.0 and Type == 'Free'")
df.query("Category in ['GAME', 'FAMILY', 'TOOLS']")

# Boolean indexing: equivalent but more verbose
df[df["Rating"] > 4.0]
df[(df["Rating"] > 4) & (df["Type"] == "Free")]  # & not 'and'

# Column selection (like dplyr::select)
df[["App", "Category", "Rating", "Type"]]      # keep columns
df.drop(columns=["Reviews", "Installs"])        # drop columns
df.filter(like="Rating")                        # pattern match
df.select_dtypes(include="number")              # by type

# .query() with variables (use @)
min_rating = 4.0
df.query("Rating > @min_rating")                # reference variable
```

.assign(): Create and Transform Columns

```
# .assign(): chainable column creation (like dplyr::mutate)
(df
    .assign(
        # Type conversion
        Reviews=lambda d: pd.to_numeric(d["Reviews"], errors="coerce
            "),
        # Log transform
        log_reviews=lambda d: np.log10(d["Reviews"] + 1),
        # String cleaning
        Size_MB=lambda d: d["Size"].str.replace("M", "").pipe(
            pd.to_numeric, errors="coerce"),
        # Date parsing
        date=lambda d: pd.to_datetime(d["Last Updated"],
            format="%B %d, %Y", errors="coerce"),
        year=lambda d: d["date"].dt.year,
        # Conditional recode (like case_when)
        price_cat=lambda d: np.select(
            [d["Price"] == 0, d["Price"] < 5, d["Price"] < 20],
            ["Free", "Under $5", "$5-$20"], default="$20+")
    )
)
```

.groupby().agg(): Aggregate per Group

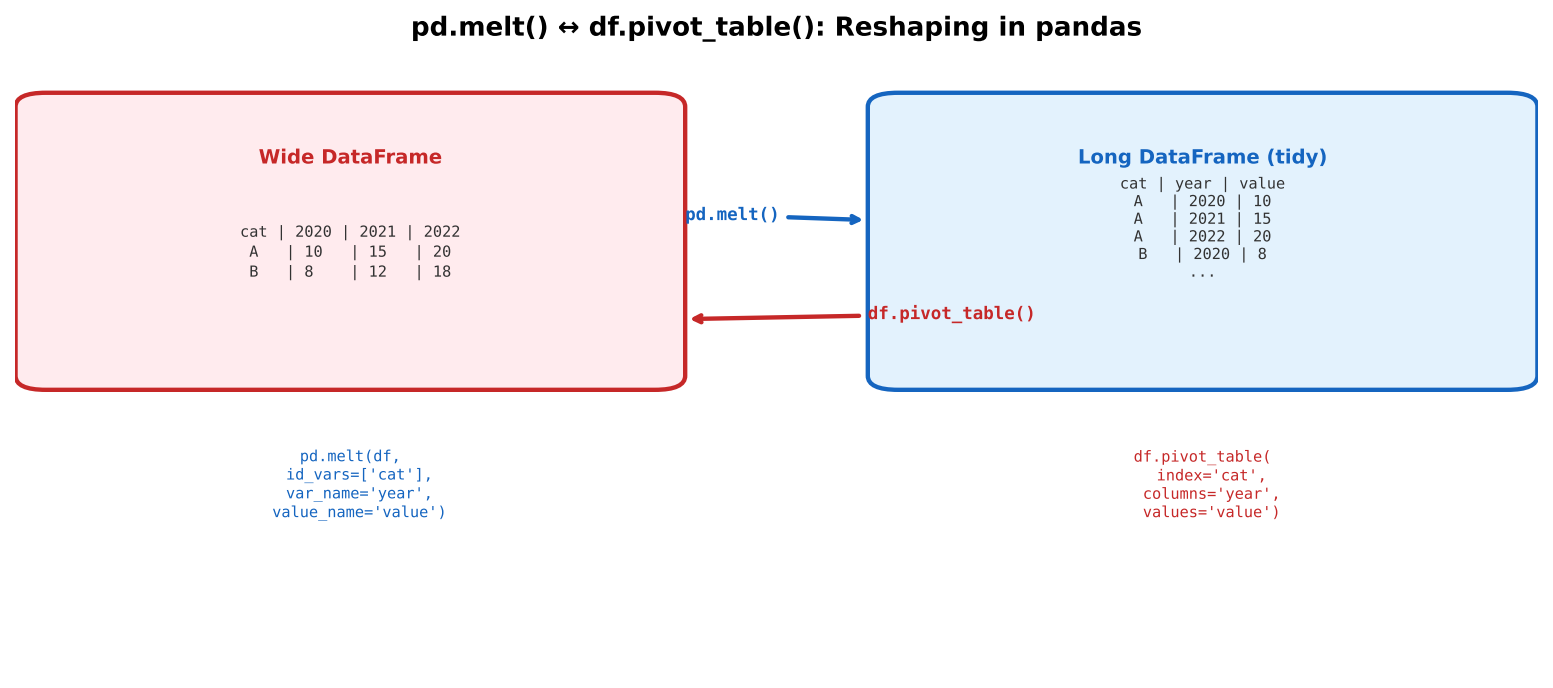
```
# Named aggregation (cleanest syntax, pandas 0.25+)
(df
    .groupby("Category")
    .agg(
        n=("Rating", "count"),
        mean_r=("Rating", "mean"),
        median_r=("Rating", "median"),
        std_r=("Rating", "std"),
        q25=("Rating", lambda x: x.quantile(0.25))
    )
    .sort_values("mean_r", ascending=False)
)

# Quick counting (like dplyr::count)
df["Category"].value_counts()                # Series, sorted desc
df.groupby("Category").size()                # same but groupby-based

# Top-n (like dplyr::slice_max)
df.nlargest(10, "Rating")                    # top 10 by Rating
summary.nlargest(8, "n")                     # top 8 by count
```

Reshaping

pd.melt() ↔ .pivot_table()



Data Insight

seaborn and plotnine require long (tidy) format, just like ggplot2. `pd.melt()` converts wide→long. `.pivot_table()` converts long→wide (for summary tables). The `.explode()` method handles multi-valued cells (e.g., Netflix's genre column).

Reshaping in Practice

```
# WIDE -> LONG (for seaborn / plotnine)
df_long = pd.melt(
    df_wide,
    id_vars=["region"],             # keep these columns
    var_name="year",                 # new col for former col names
    value_name="revenue"             # new col for values
)
df_long["year"] = df_long["year"].astype(int)  # string -> int

# LONG -> WIDE (for summary tables)
df_wide = df_long.pivot_table(
    index="region",
    columns="year",
    values="revenue",
    aggfunc="sum"                    # handles duplicates
)

# EXPLODE: one row per value in a list column
netflix["genres"] = netflix["listed_in"].str.split(", ")
netflix_exploded = netflix.explode("genres")  # like separate_rows()
```

Cleaning Patterns

Six Common Cleaning Patterns

Six Common Data Cleaning Patterns in Python (pandas)

Pattern	Python Code	Purpose
Missing values	`df.dropna(subset=['Rating'])` `df['col'].fillna(0)`	Remove or impute
Type conversion	`pd.to_numeric(df['Reviews'],` `  errors='coerce')`	String → numeric
String cleaning	`df['Size'].str.replace('M','')` `  .astype(float)`	Remove units, cast
Date parsing	`pd.to_datetime(df['date'],` `  format='%B %d, %Y')`	Parse dates
Recoding	`np.where(df['Price']==0,` `  'Free','Paid')`	Create categories
Multi-value explode	`df['genres'].str.split(', ')` `  .explode()`	One row per value

Cleaning the Google Play Store Dataset

```
apps_clean = (
    pd.read_csv("googleplaystore.csv")
    .drop_duplicates(subset="App", keep="first")
    .assign(
        Reviews=lambda d: pd.to_numeric(d["Reviews"], errors="coerce
            "),
        Installs=lambda d: (d["Installs"]
            .str.replace(r"[+,]", "", regex=True)
            .pipe(pd.to_numeric, errors="coerce")),
        Size_MB=lambda d: (d["Size"]
            .str.replace("M", "").str.replace("k", "e-3")
            .pipe(pd.to_numeric, errors="coerce")),
        Price=lambda d: (d["Price"]
            .str.replace("$", "", regex=False)
            .pipe(pd.to_numeric, errors="coerce")),
        date=lambda d: pd.to_datetime(
            d["Last Updated"], format="%B %d, %Y", errors="coerce")
    )
    .dropna(subset=["Rating"])
    .query("Type in ['Free', 'Paid']")
    .query("Rating >= 1 and Rating <= 5")
)
```

Joins

pd.merge(): Combining DataFrames

```
# LEFT JOIN (most common for viz)
result = pd.merge(
    apps, lookup_table,
    on="Category",            # join key
    how="left"                # keep all apps rows
)

# INNER JOIN
pd.merge(apps, reviews, on="App", how="inner")

# ANTI JOIN (rows in apps with NO match in reviews)
anti = apps[~apps["App"].isin(reviews["App"])]
# Or with merge:
merged = pd.merge(apps, reviews, on="App", how="left", indicator=
    True)
anti = merged.query("_merge == 'left_only'").drop(columns="_merge")

# JOIN TYPES: how = "left", "right", "inner", "outer"
```

Pro-Tip

pandas uses `pd.merge()` or `df.merge()` with `how=` parameter. For anti-joins, use `indicator=True` then filter on `_merge == 'left_only'`, or simply use `~df["key"].isin(other["key"])`.

Full Chain

One Expression: CSV → Chart

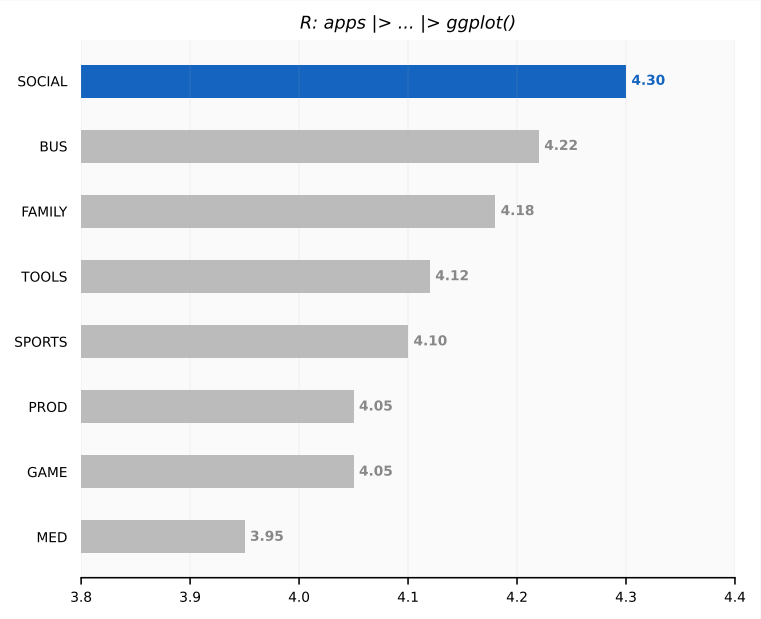
Full Chain: One Expression from CSV to Chart (Zero Variables)

```
(pd.read_csv("googleplaystore.csv")
    .drop_duplicates(subset="App")
                .assign(
Reviews=lambda d: pd.to_numeric(d["Reviews"], errors="coerce"),
Rating=lambda d: d["Rating"].where(d["Rating"].between(1, 5)))
            .dropna(subset=["Rating"])
        .query("Type in ['Free', 'Paid']")
                .groupby("Category")
  .agg(mean_r=("Rating", "mean"), n=("Rating", "count"))
                  .nlargest(8, "n")
                .sort_values("mean_r")
  .plot.barh(y="mean_r", color="#1565C0", figsize=(6,5))
                          )
```

Complete Examples

R: Pipe-to-Plot (Reference)

```r
# R equivalent for comparison
apps_clean |>
  group_by(Category) |>
  summarise(
    mean_r = mean(Rating),
    n = n(),
    .groups = "drop") |>
  slice_max(n, n = 8) |>
  mutate(Category =
    fct_reorder(Category, mean_r),
    hl = mean_r == max(mean_r)) |>
  ggplot(aes(x = Category,
    y = mean_r, fill = hl)) +
  geom_col(width = 0.5,
    show.legend = FALSE) +
  scale_fill_manual(values =
    c("TRUE" = "#1565C0",
      "FALSE" = "#BBBBBB")) +
  coord_flip() +
  theme_minimal() +
  labs(title = "R: pipe-to-plot")
```



Python: Chain-to-Plot

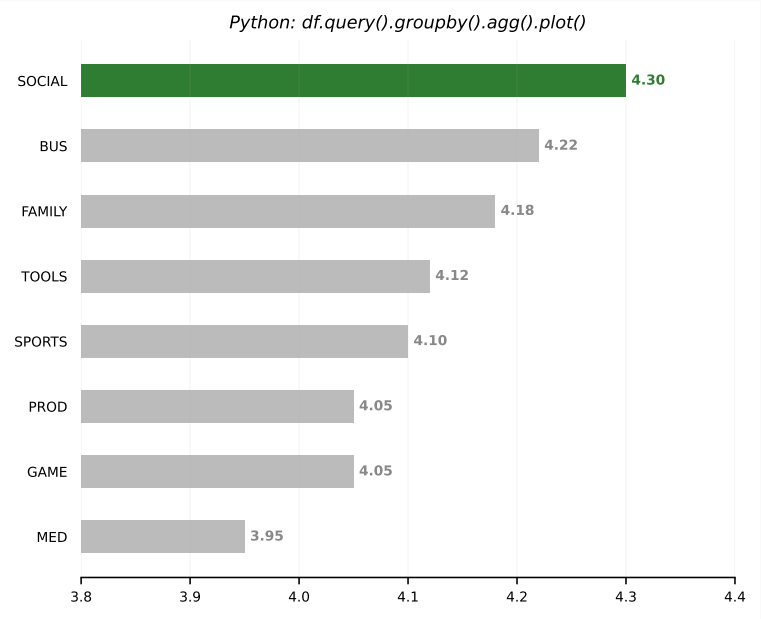
```python
# Python chain equivalent
summary = (apps_clean
    .groupby("Category")
    .agg(
        mean_r=("Rating", "mean"),
        n=("Rating", "count"))
    .nlargest(8, "n")
    .sort_values("mean_r"))

fig, ax = plt.subplots(
    figsize=(6, 5))
colors = ["#BBBBBB"] * len(summary)
colors[-1] = "#2E7D32"  # accent

ax.barh(summary.index,
    summary["mean_r"],
    color=colors, height=0.5)

for i, (idx, row) in enumerate(
    summary.iterrows()):
    ax.text(row["mean_r"] + 0.005,
        i, f"{row['mean_r']:.2f}",
        va="center", fontsize=7)

ax.set_xlim(3.8, 4.4)
ax.set_title("Python: chain-to-plot
```



Synthesis

Key Takeaways

1. **Method chaining** (parenthesised multi-line) is pandas' equivalent of R's |> pipe. Wrap in `(...)`.
2. **.assign()** = `mutate()`; **.query()** = `filter()`; **.groupby().agg()** = `group_by() + summarise()`.
3. **pd.melt()** = `pivot_longer()`; **.pivot_table()** = `pivot_wider()`; **.explode()** = `separate_rows()`.
4. Common cleaning: `pd.to_numeric(errors='coerce')`, `.str.replace()`, `pd.to_datetime()`, `np.where()` / `np.select()`.
5. **Full chain**: one parenthesised expression from `pd.read_csv()` to `.plot.barh()` — zero intermediate variables.
6. For anti-joins: `~df["key"].isin(other["key"])` or `merge(indicator=True)`.

Essential Readings

- McKinney, W. (2022). *Python for Data Analysis*, 3rd ed., Chapters 5, 7–8 (pandas, Data Wrangling).
- pandas documentation: Method chaining style guide. `https://pandas.pydata.org/docs/`
- VanderPlas, J. (2016). *Python Data Science Handbook*, Chapter 3 (pandas).

Next Module: Missing Data Visualization (W04-M04)